

Stock Price Prediction

Frankli Márk Bánházi Dorina

May 2025

1 Introduction

Stock price prediction is a long-standing problem in quantitative finance, attracting interest from researchers, traders, and data scientists due to its economic significance and inherent complexity. Financial markets are influenced by a wide range of factors, including macroeconomic indicators, investor sentiment, and market microstructure, making price movements difficult to model. Moreover, stock prices often exhibit noise, non-stationarity, and volatility, posing additional challenges for predictive modeling.

In this project, we explore the problem of stock price prediction and trading strategy design from both traditional and modern machine learning perspectives. We begin with exploratory data analysis (EDA) to understand the structure and dynamics of historical stock price data. Following this, we backtest a range of well-known technical analysis-based strategies, including Moving Average Convergence Divergence (MACD), Relative Strength Index (RSI), Exponential Moving Average (EMA) and Bollinger Bands (BB). These rule-based strategies serve as interpretable baselines and provide insight into the market behavior under different heuristics.

Building on this foundation, we implement and compare several reinforcement learning (RL) algorithms for automated trading. Specifically, we focus on Deep Q-Network (DQN), Proximal Policy Optimization (PPO), and Deep Deterministic Policy Gradient (DDPG), each representing a different class of RL methods. These agents are trained to learn trading policies by interacting with a simulated trading environment, aiming to maximize cumulative returns.

Throughout the project, we evaluate and compare the performance of traditional strategies and RL-based agents using backtesting on historical data. This comprehensive approach allows us to assess the practical viability and relative strengths of each method in the context of financial decision-making.

2 Dataset

The dataset used in this project is sourced from Kaggle¹ and contains comprehensive historical daily stock market data. It covers all U.S.-based stocks and exchange-traded funds (ETFs) traded on the NYSE, NASDAQ, and NYSE MKT.

Each stock or ETF is represented by a separate CSV file containing the following columns:

- **Date:** The trading date in YYYY-MM-DD format.
- **Open:** The price at which the security first traded when the market opened.
- **High:** The highest price reached during the trading day.
- **Low:** The lowest price reached during the trading day.
- **Close:** The final price at which the security traded before the market closed.
- **Volume:** The total number of shares traded during the day.
- **OpenInt:** The open interest, typically used in derivatives markets. This column is included in the dataset but was not used in our analysis.

All prices in the dataset have been adjusted for dividends and stock splits, ensuring consistency in time-series modeling and preventing artificial jumps in price due to corporate actions.

3 Exploratory Data Analysis

Before applying any trading strategies or machine learning models, we performed a series of exploratory data analysis (EDA) steps to clean and better understand the dataset.

3.1 Preprocessing

Our first step involved filtering out unusable data files. Specifically, we excluded:

- Empty or corrupted files,
- Files where any of the price columns (Open, High, Low, Close) contained zero values,
- Files with fewer than 300 rows, which corresponds to approximately ten months of trading data.

After preprocessing, we retained a total of 4,571 valid stock time series and 1,155 ETF time series.

¹<https://www.kaggle.com/datasets/borismarjanovic/price-volume-data-for-all-us-stocks-etfs/data>

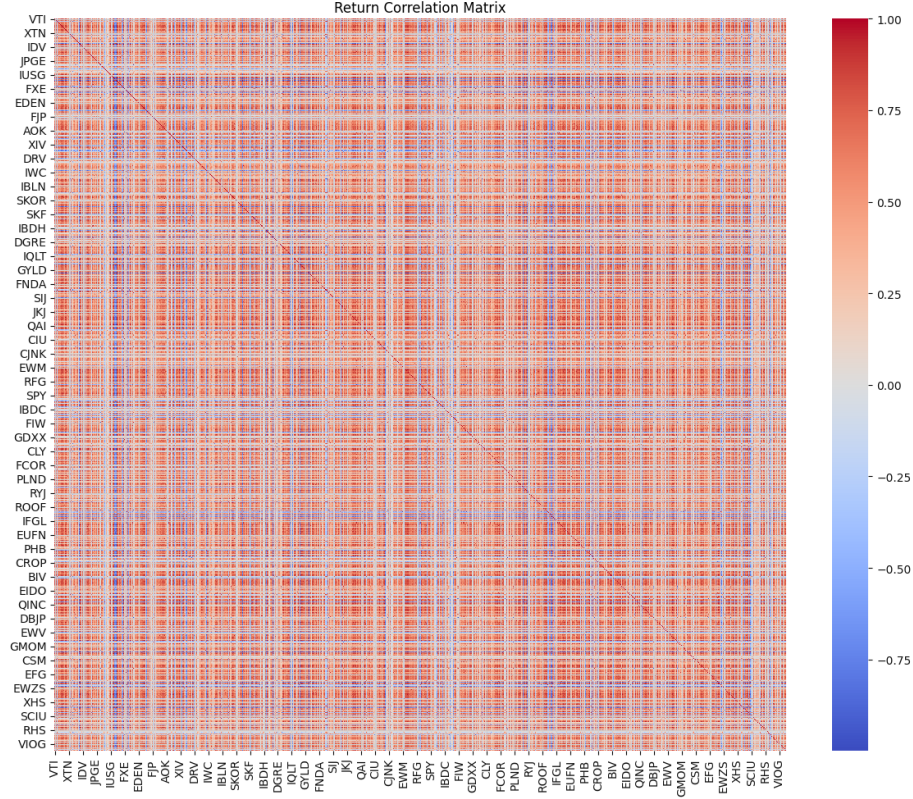


Figure 1: Correlation Matrix of ETFs

3.2 Correlation Analysis

To understand the relationships between different assets, we computed and visualized the correlation matrices of the closing prices for both stocks and ETFs. As expected, ETFs exhibited much higher overall correlation than individual stocks. This is consistent with the nature of ETFs, which are typically constructed as diversified baskets of underlying equities—leading to more synchronized price movements.

3.3 Dimensionality Reduction and Clustering

We also applied Principal Component Analysis (PCA) followed by K-means clustering on the closing prices to uncover patterns in asset behavior. This revealed meaningful clusters of stocks and ETFs with similar price dynamics. Two assets—CXDC and MTBCP—stood out as clear outliers in the PCA projection, with component scores in the thousands, while most others ranged between 0 and 10. Upon inspection, both had very low trading volume and unusually high

- (10, 30)
- (15, 50)
- (20, 50)
- (50, 100)
- (50, 200)
- **RSI (Relative Strength Index):** Five parameter tuples representing (period, lower threshold, upper threshold):
 - (14, 30, 70)
 - (9, 25, 75)
 - (5, 20, 80)
 - (21, 35, 65)
 - (28, 40, 60)
- **Bollinger Bands:** Five parameter tuples representing (window length, standard deviation multiplier):
 - (20, 2.0)
 - (10, 1.5)
 - (14, 1.5)
 - (50, 2.0)
 - (100, 2.0)
- **Combined MACD and RSI Strategy:** Five parameter tuples combining MACD (short, long, signal) and RSI (period, lower, upper) parameters:
 - (12, 26, 9, 14, 30, 70)
 - (5, 35, 5, 10, 25, 75)
 - (8, 17, 9, 7, 20, 80)
 - (10, 30, 10, 20, 40, 60)
 - (6, 19, 3, 5, 28, 72)
- **Buy & Hold Strategy**

Due to computational limitations, we restricted the backtesting to the first 10 stocks and ETFs in the dataset to ensure manageable runtime and resource use. The initial cash was set to \$1 million.

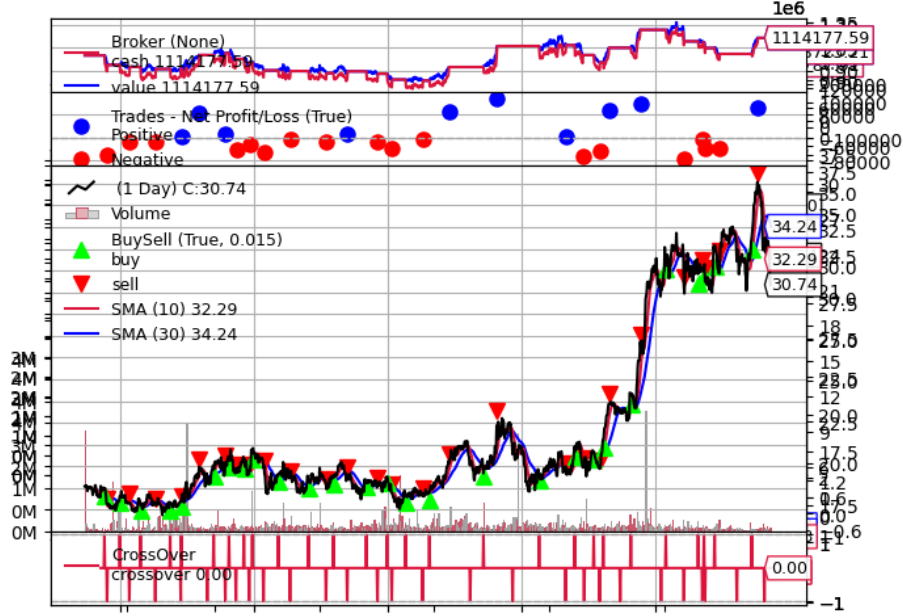


Figure 3: Backtested stock with a 10% overall profit

5 Reinforcement Learning Algorithms

5.1 Deep Q-Network (DQN)

Deep Q-Network (DQN) is an algorithm that combines Q-learning with deep neural networks to approximate the optimal action-value function. This approach enables an agent to learn policies directly from high-dimensional inputs by using experience replay and a target network to stabilize training and improve convergence.

5.1.1 Implementation Details

A DQN agent was implemented for trading stocks and ETFs using a custom OpenAI Gym environment. The environment's state consists of a sliding window of recent closing prices, while the action space includes three discrete actions: hold, buy, and sell.

Environment

- **State space:** A vector of recent closing prices with a window size of 10.
- **Action space:** Discrete with three actions — 0: hold, 1: buy, 2: sell.

Table 1: Backtesting Results Summary (Stocks)

	Name	Strategy	Parameters	Value	Diff	Interval
2	IBA	macd	(3, 10, 16)	7.133005e+06	613.3%	4641 days
198	WTI	macd	(12, 26, 5)	5.821220e+06	482.12%	4641 days
185	GPK	bollinger_bands	(10, 1.5)	3.693493e+06	269.35%	4641 days
221	MYGN	macd	(8, 17, 9)	3.548561e+06	254.86%	4641 days
5	IBA	sma_cross	(10, 30)	3.372536e+06	237.25%	4641 days
:	:	:	:	:	:	:
89	NCS	rsi	(9, 25, 75)	1.835185e+05	-81.65%	4641 days
168	GPK	macd	(5, 35, 5)	1.267153e+05	-87.33%	4641 days
88	NCS	rsi	(14, 30, 70)	1.091325e+05	-89.09%	4641 days
103	NCS	buy_and_hold	-	8.803200e+04	-91.2%	4641 days
211	WTI	bollinger_bands	(10, 1.5)	5.167667e+04	-94.83%	4641 days

- **Reward:** Dense reward defined as the change in portfolio value (cash plus shares times current price) after each step.
- **Initial cash:** Set to the constant `INIT_CASH` (e.g., \$1 million).
- **Episode termination:** Occurs when the end of the price series is reached.

DQN Architecture The Q-network is a fully connected feedforward neural network composed of:

- Input layer of size equal to the window size (10).
- Two hidden layers, each with a hyperparameter-defined number of units (`hidden_dim`) and ReLU activations.
- Output layer of size 3 (corresponding to the number of actions).

Agent Design The agent maintains two networks:

- A **primary Q-network** used to select actions and learn Q-values.
- A **target network** that is updated periodically to provide stable target values during learning.

The agent uses an ϵ -greedy strategy for exploration. Experiences are stored in a replay buffer and sampled in batches to update the Q-network using the Bellman equation.

Table 2: Backtesting Results Summary (ETFs)

	Name	Strategy	Parameters	Value	Diff	Interval
17	VTI	bollinger_bands	(10, 1.5)	3,077,872.043	207.79%	4641 days
167	PEJ	buy_and_hold	-	2,944,858.000	194.49%	4523 days
16	VTI	bollinger_bands	(14, 1.5)	2,906,157.168	190.62%	4641 days
77	VIOV	buy_and_hold	-	2,645,342.000	164.53%	2614 days
25	VTI	buy_and_hold	-	2,541,854.000	154.19%	4641 days
	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
179	RUSL	rsi	(9, 25, 75)	289,737.187	-71.03%	2359 days
184	RUSL	bollinger_bands	(14, 1.5)	289,209.459	-71.08%	2359 days
170	RUSL	macd	(3, 10, 16)	243,243.371	-75.68%	2359 days
180	RUSL	rsi	(5, 20, 80)	202,626.107	-79.74%	2359 days
168	RUSL	macd	(5, 35, 5)	143,435.109	-85.66%	2359 days

Agent Hyperparameters Hyperparameters were optimized using Optuna, with the following search spaces:

Hyperparameter	Search Range/Values	Description
γ (discount factor)	0.85 to 0.99	Discount factor for future rewards
Learning rate (lr)	10^{-4} to 10^{-2} (log scale)	Adam optimizer learning rate
Epsilon decay	0.9 to 0.999	Decay rate for epsilon-greedy policy
Hidden dimension ($hidden_dim$)	32 to 64 (integer)	Units per hidden layer
Batch size	32 or 64	Mini-batch size for replay training
Replay memory size	2000 to 5000	Capacity of experience replay buffer

Table 3: Hyperparameters tuned for the DQN agent

Training Procedure

- Number of episodes per Optuna trial: 5 (to allow fast tuning).
- Transitions of the form $(s, a, r, s', done)$ are stored in a replay buffer.
- Mini-batches are sampled to update the Q-network via mean squared error loss.
- The target network is synchronized with the Q-network at the end of each episode.
- Exploration uses an ϵ -greedy policy, with ϵ initialized at 1.0 and decayed toward a minimum of 0.01.

Evaluation Backtesting was conducted on multiple assets, limited by the constant `RL_LIMIT`. For each asset:

- Hyperparameter optimization was performed using 10 Optuna trials.
- The best parameters were used to instantiate a new agent for evaluation.
- The agent’s portfolio value was tracked over the episode to visualize trading performance.

Visualization

- For each asset, a line plot shows the portfolio value progression over time using the best agent from optimization.
- A combined plot overlays results from all assets to assess consistency.
- If available, Optuna’s diagnostics (optimization history and parameter importances) are plotted for insight into the tuning process.

5.2 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a policy gradient method that improves stability and performance by using a clipped surrogate objective function. PPO strikes a balance between efficient policy updates and the risk of performance collapse caused by large policy changes, making it well-suited for continuous training in reinforcement learning tasks such as trading.

5.2.1 Implementation Details

A PPO agent was implemented for stock and ETF trading using the same custom OpenAI Gym environment as the DQN agent. The agent learns to maximize returns by directly optimizing a stochastic policy represented by a neural network.

Environment

- **State space:** A vector of recent closing prices with a window size of 10.
- **Action space:** Discrete with three actions — 0: hold, 1: buy, 2: sell.
- **Reward:** Dense reward equal to the change in portfolio value after each action.
- **Initial cash:** Set to a fixed constant (e.g., \$1 million).
- **Episode termination:** Occurs at the end of the price series.

PPO Architecture The agent consists of two separate neural networks:

- **Actor:** Outputs a probability distribution over actions using a softmax layer. It is a feedforward neural network with two hidden layers of configurable size.
- **Critic:** Outputs a scalar estimate of the state value function $V(s)$ using a similar feedforward structure.

Both networks use ReLU activation functions between layers. The number of hidden units and layers are hyperparameters subject to tuning.

Agent Design

- The actor samples actions from a categorical distribution during training.
- The critic evaluates the state values used for computing advantages.
- Returns are computed using discounted future rewards, adjusted for terminal states.

Agent Hyperparameters Hyperparameters were tuned using Optuna over the following search spaces:

Hyperparameter	Search Range/Values	Description
γ (discount factor)	0.85 to 0.999	Discount factor for future rewards
Clip ratio (ϵ)	0.1 to 0.3	Controls extent of policy updates
Learning rate (lr)	10^{-5} to 10^{-3} (log scale)	Learning rate for both actor and critic
Hidden dimension	32 to 128 (integer)	Units per hidden layer
Hidden layers	1 or 2	Number of hidden layers

Table 4: Hyperparameters tuned for the PPO agent

Training Procedure

- For each Optuna trial, the agent was trained for a fixed number of episodes.
- During training, the agent collects full trajectories consisting of $(s, a, \log \pi(a|s), r, s', \text{done})$.
- Returns are computed using bootstrapped estimates from the critic for non-terminal states.
- Advantages are calculated as the difference between returns and the critic’s value estimates.
- The actor and critic networks are updated using gradient descent with separate Adam optimizers.

Evaluation Backtesting was conducted on multiple assets (limited by `RL_LIMIT`). For each asset:

- Optuna was run for 10 trials to find the best hyperparameters.
- The final agent was retrained using these hyperparameters.
- The portfolio value was tracked and plotted over the episode to assess trading performance.

Visualization

- For each asset, a line plot shows the portfolio value over time using the best PPO agent.
- A combined plot visualizes performance across assets for comparative analysis.
- When available, Optuna’s optimization history and parameter importance plots were displayed to diagnose the tuning process.

5.3 Deep Deterministic Policy Gradient (DDPG)

Deep Deterministic Policy Gradient (DDPG) is an actor-critic algorithm designed for continuous action spaces. In this work, it is adapted to a discrete action space using softmax outputs, allowing it to be applied to financial trading tasks. DDPG utilizes two networks (actor and critic) along with their corresponding target networks, and updates parameters using soft target updates. The deterministic nature of the policy enables efficient learning in high-dimensional action spaces, while the experience replay buffer helps stabilize training.

5.3.1 Implementation Details

A DDPG agent was implemented to operate in the same custom OpenAI Gym environment as the DQN and PPO agents. The agent learns a deterministic policy to map states to actions using an actor network, while the critic estimates the Q-values of state-action pairs.

Environment

- **State space:** A vector of recent closing prices with a window size of 10.
- **Action space:** Discrete with three actions — 0: hold, 1: buy, 2: sell.
- **Reward:** Dense reward equal to the change in portfolio value after each action.
- **Initial cash:** Set to a fixed constant (e.g., \$1 million).
- **Episode termination:** Occurs at the end of the price series.

DDPG Architecture

- **Actor:** A feedforward neural network with two hidden layers of 64 units each, using ReLU activations. The output is a probability distribution over actions obtained via softmax.
- **Critic:** A feedforward network with the same architecture that takes as input the concatenated state and one-hot encoded action vector, and outputs a scalar Q-value.
- **Target Networks:** Separate target actor and critic networks are maintained and updated slowly using Polyak averaging.

Agent Design

- Actions are sampled from the actor’s output distribution.
- Experiences $(s, a, r, s', \text{done})$ are stored in a replay buffer and sampled randomly during training.
- The critic is trained to minimize the temporal difference error between the predicted Q-value and the target Q-value.
- The actor is trained to maximize the expected Q-value estimated by the critic.
- Target networks are updated using soft updates.

Agent Hyperparameters Hyperparameters were tuned using Optuna over the following search spaces:

Hyperparameter	Search Range/Values	Description
γ (discount factor)	0.85 to 0.999	Discount factor for future rewards
τ (soft update rate)	0.001 to 0.01	Rate for updating target networks
lr_{actor}	10^{-5} to 10^{-3} (log scale)	Learning rate for the actor
$\text{lr}_{\text{critic}}$	10^{-5} to 10^{-3} (log scale)	Learning rate for the critic
Batch size	16, 32, or 64	Number of samples per training update
Replay buffer size	2000 to 10000	Capacity of the experience replay buffer

Table 5: Hyperparameters tuned for the DDPG agent

Training Procedure

- For each Optuna trial, the agent was trained over multiple episodes.
- At each step, actions were selected from the actor’s output, and transitions were added to the replay buffer.
- Training occurred after sufficient experience was collected.

- The critic was trained to minimize temporal difference loss using bootstrapped target Q-values.
- The actor was trained to maximize expected Q-values.
- Target networks were softly updated after each training step.

Evaluation

- Optuna was used to perform 10 trials per asset.
- The best hyperparameters were used to retrain the agent for final evaluation.
- Portfolio values were tracked and plotted over time to assess performance.

Visualization

- For each asset, a line plot shows portfolio value over time using the best DDPG agent.
- A combined plot visualizes comparative performance across all assets.
- When available, Optuna optimization history and parameter importance plots were displayed.

6 Conclusion

Due to computational limitations, we limited our experiments to two individual stocks: IBA (Industrias Bachoco) and FLIC (First of Long Island Corporation).

Traditional technical indicator-based strategies served as benchmarks. For IBA, the best-performing backtest returned an exceptional 613.3%, using a MACD strategy with parameters (3, 10, 16). However, this result was an outlier; all other traditional strategies yielded returns in the range of 220–230%. For FLIC, the best result came from a simple *buy-and-hold* strategy, yielding a return of 232.39%.

In contrast, the reinforcement learning agents underperformed. None of the RL-based methods reliably outperformed the best classical strategies, and in some cases—such as with the DDPG agent—the final portfolio closed with a loss. These results highlight the challenges of applying RL to financial trading, particularly in low-dimensional, high-noise environments with limited training data.

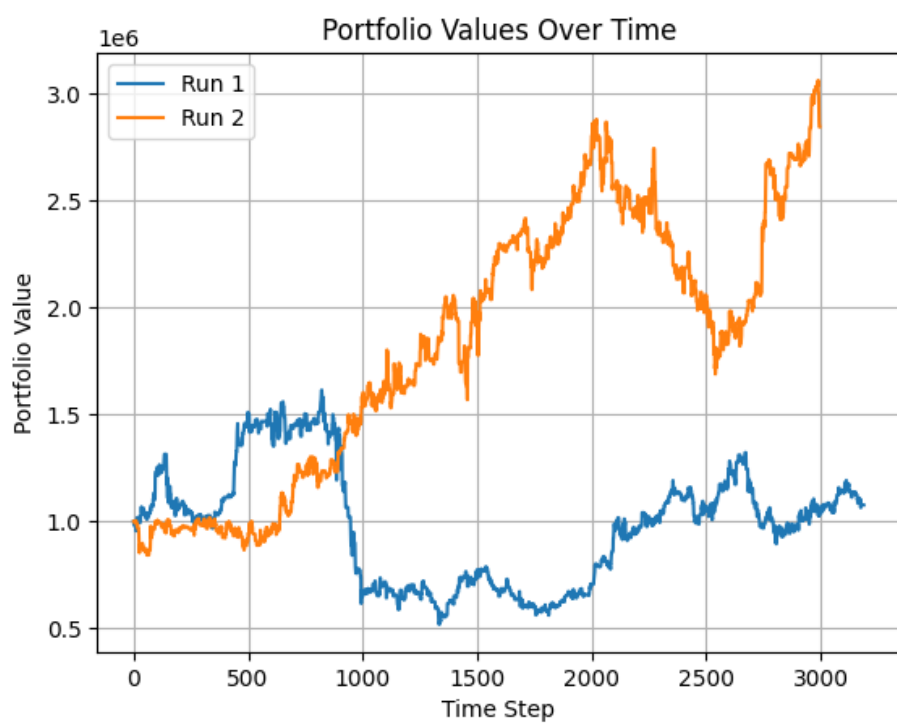


Figure 4: DQN's performance for two stocks

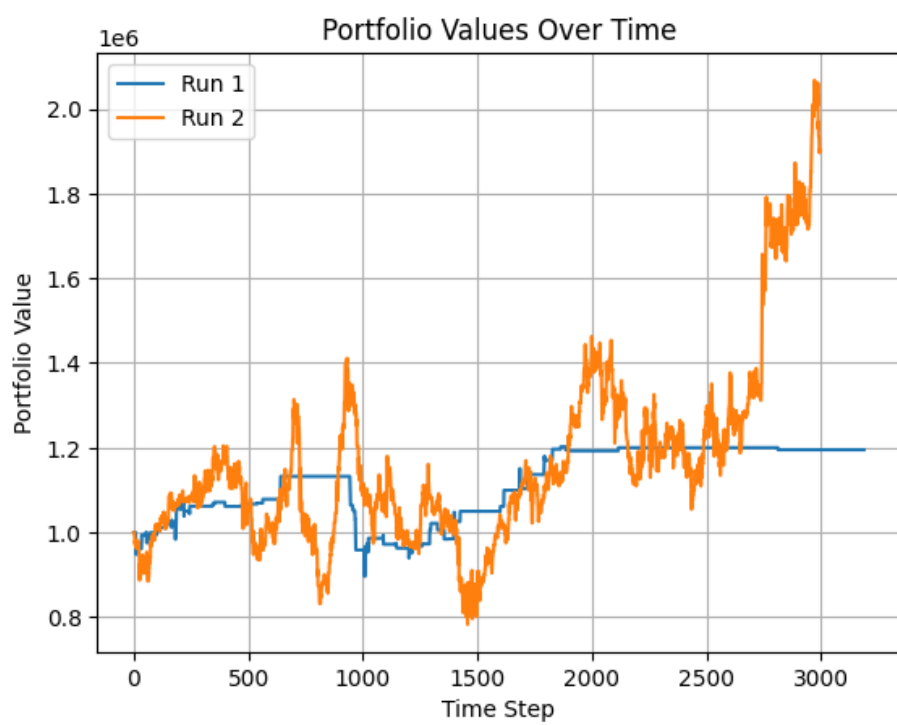


Figure 5: PPO's performance for two stocks

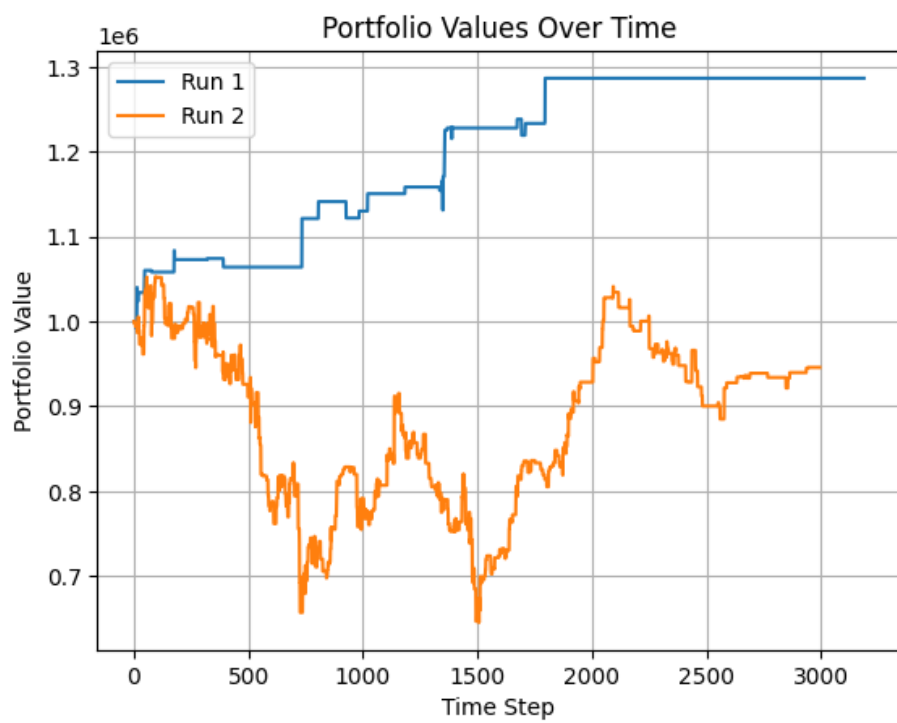


Figure 6: DDPG's performance for two stocks